Software Construction and Testing Project, Winter 2026

Milestone 1: Idea & Architecture

Closet, Laundry and Outfit Management System

Team: Runtime Terror | TA: Nadeen Serag

Mohamed Amro (16003231), Valen George (16002220), Joudy Khaled (16001246), Malak Mohamed (16001972), Ziad Mohamed (16003070), Abdelrahman Ahmed (16008929)

1. Project Idea

Project idea. The Closet, Laundry and Outfit Management System is a software application that helps users organize their wardrobes, manage laundry activities, and plan outfits. It provides one centralized digital platform for clothing information, laundry status, and outfit coordination.

Objectives. Simplify wardrobe organization, reduce the time needed to find suitable clothing, support laundry tracking, and manage clothing-related activities in one place, with an intuitive experience on mobile and desktop devices.

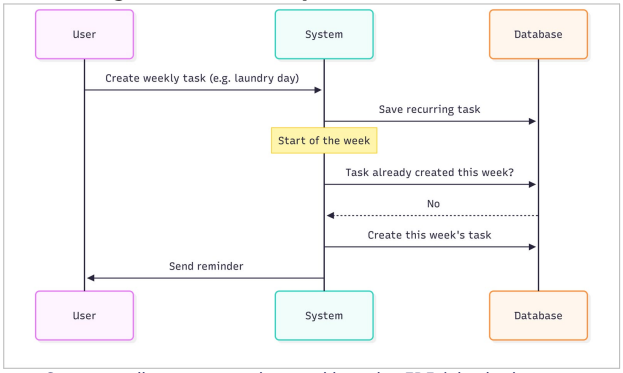
Expected outcome. A functional application where users maintain a digital closet, add and filter clothing items, track laundry progress, and organize outfits. Outfit matching and weekly task reminders support users' daily wardrobe routines.

Justification. Managing clothing, remembering laundry status, and deciding what to wear is time-consuming when done manually. Integrating these activities makes wardrobe management more organized and efficient, and the modular design lets features work together while each part keeps clear responsibilities.

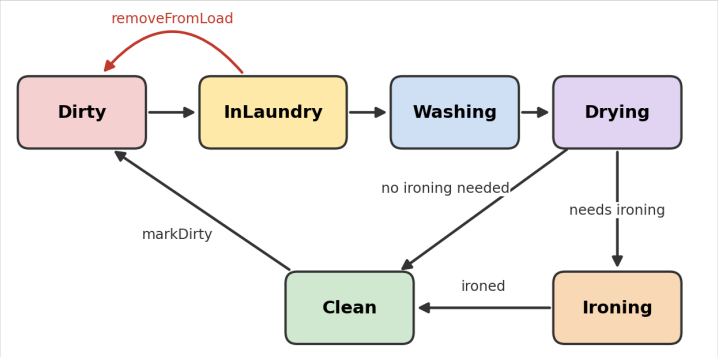
2. Requirements and Core User Stories

ID	Module (owner)	Requirement	Core user story
Functional requirements			
FR1	Laundry	**Laundry lifecycle tracking.** The user marks closet items as dirty and groups them into laundry loads by color and fabric. Each load advances Washing → Drying → Clean. When a load finishes, items return to the closet as clean, except items whose fabric needs ironing, which are handed to Ironing (FR4).	As a user, I want to group my dirty clothes into loads by color and fabric and track each load until it is clean, so that I know what is ready to wear and avoid damaging clothes by mixing them.
FR2	Outfits & Matching	**Outfit building and suggestions.** The system lets users build outfits manually from clean closet items, and suggests outfits for a chosen occasion and season, ranked by color compatibility. Dirty items and items being ironed are never used.	As a user, I want outfit suggestions for a chosen occasion and season, built only from my clean clothes, so that I can get dressed quickly without checking the laundry basket.
FR3	Wear Log	**Wear log and stats.** The user can log the outfits they wore. The system shows the most worn and least worn items, items not worn in the last 90 days, and a weekly summary.	As a user, I want to log what I wear and see my wear stats, so that I know which clothes I wear the most and which ones I never wear.
FR4	Ironing, Care & Auth	**Ironing queue and care instructions.** When a laundry load finishes, items whose fabric needs ironing go to the ironing queue instead of back to the closet. Laundry returns all other items as clean. Each queued item shows care instructions for its fabric, such as iron temperature. When the user marks an item as ironed, it moves from the queue to clean.	As a user, I want to see which of my washed clothes need ironing and at what temperature, so that I can iron them without damaging them.
FR5	Recurring Tasks	**Recurring weekly tasks.** Users define weekly recurring tasks (laundry day, ironing day, closet check). The system generates them automatically each week and sends reminders.	As a user, I want weekly laundry and ironing reminders so that chores don't pile up.
Non-functional requirements			
NFR1	Laundry	**Data consistency.** An item belongs to at most one active load at any time, and the system rejects any invalid state transition (e.g., Dirty → Drying) with an error, leaving the data unchanged. Automated tests verify that 100% of invalid transitions are rejected.	
NFR2	Outfits & Matching	**Extensibility.** The matching engine follows the Open/Closed Principle: new matching criteria and algorithms can be added without modifying existing code.	
NFR3	Wear Log	**Performance.** Reports are generated in under 3 seconds for 1,000 wear logs.	
NFR4	Ironing, Care & Auth	**Security.** Passwords are stored hashed, logged-in users are identified by a token, and each user can only see and edit their own items and ironing queue.	
NFR5	Recurring Tasks	**Reliability.** A recurring task is never generated twice for the same period, and generation continues correctly after a system restart.	

Bonus diagrams for the requirements



Sequence diagram: recurring weekly tasks, FR5 (also in the repo: docs/sequence-diagram.png)



State diagram: laundry item status, FR1 / NFR1 (State pattern)

3. System Architecture: Layered Monolith

Justification. A layered monolith fits because the closet, laundry, outfit, wear-log, ironing and task features share the same clothing items and one database, so separate services would add complexity without benefit. Each layer has one job (UI, request handling, business rules, data access), which keeps the code easy to test layer by layer. The business logic of every module lives in the service layer. Authentication runs before every request, so all modules are protected. Laundry and Ironing never overlap: Laundry hands items needing ironing over by setting their status to “ironing”, and only the Ironing module moves them to clean.

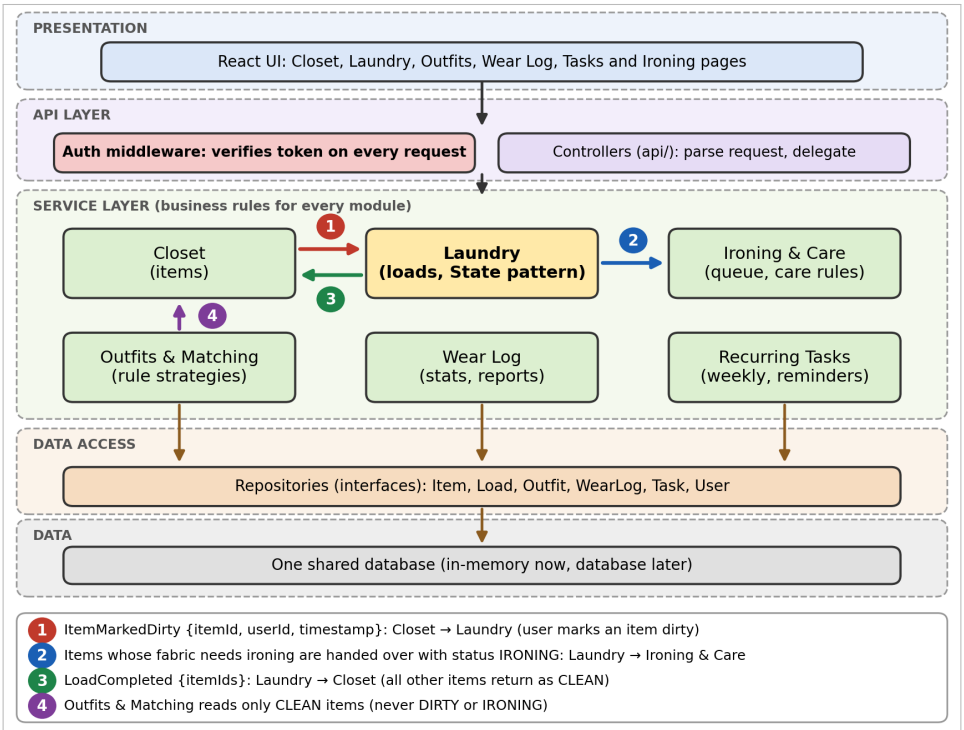
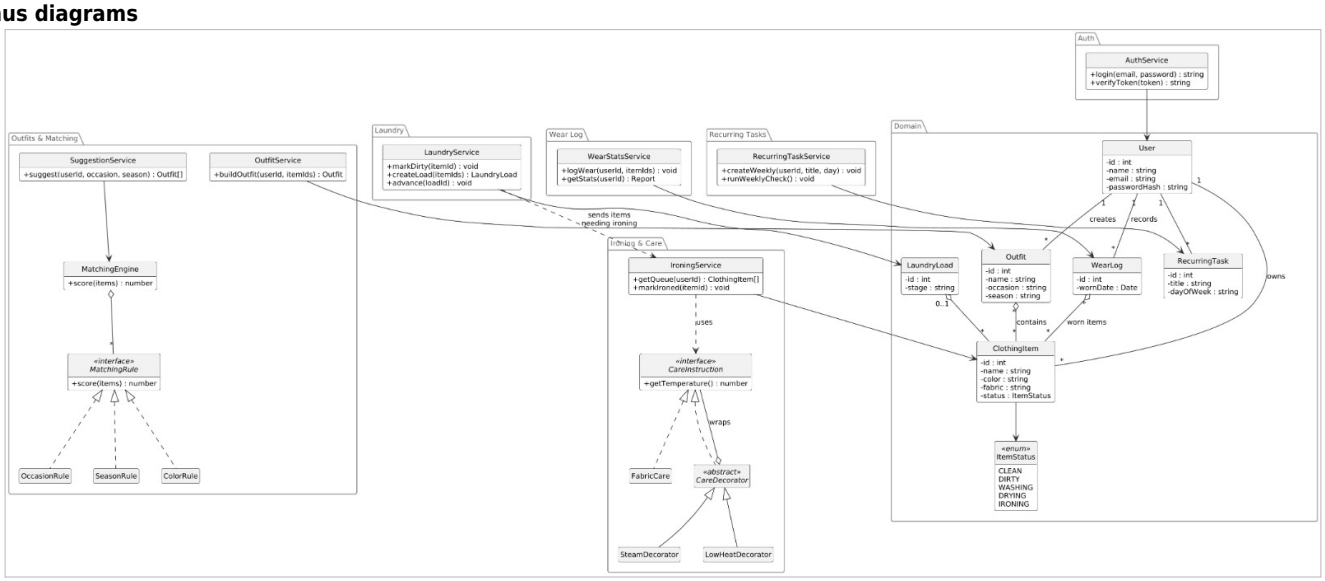
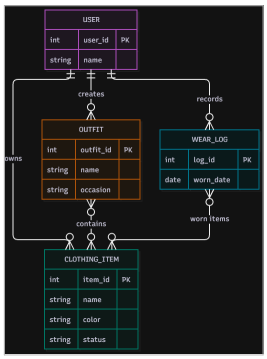


Figure 1: main components, interactions and data flow between modules (numbers 1–4 are explained in the legend)

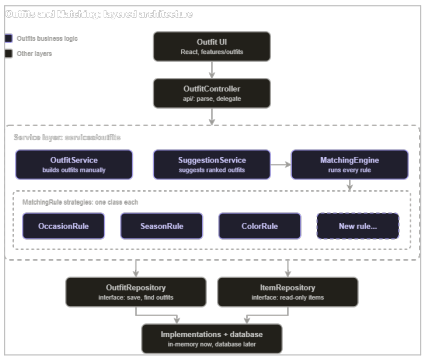
Bonus diagrams



Class diagram



Entity-relationship diagram



Outfits module: layered architecture